

CSE5026: Introduction and Frontiers of Cognitive Science

Neuromorphic Computing

Yang Xu 徐炆

Department of Computer Science and Engineering
Southern University of Science and Technology

Overview

- Introduction to Spiking Neural Networks (SNN)
- Leaky Integrate-and-Fire (LIF) Neurons
- Spike-Timing Dependent Plasticity (STDP)

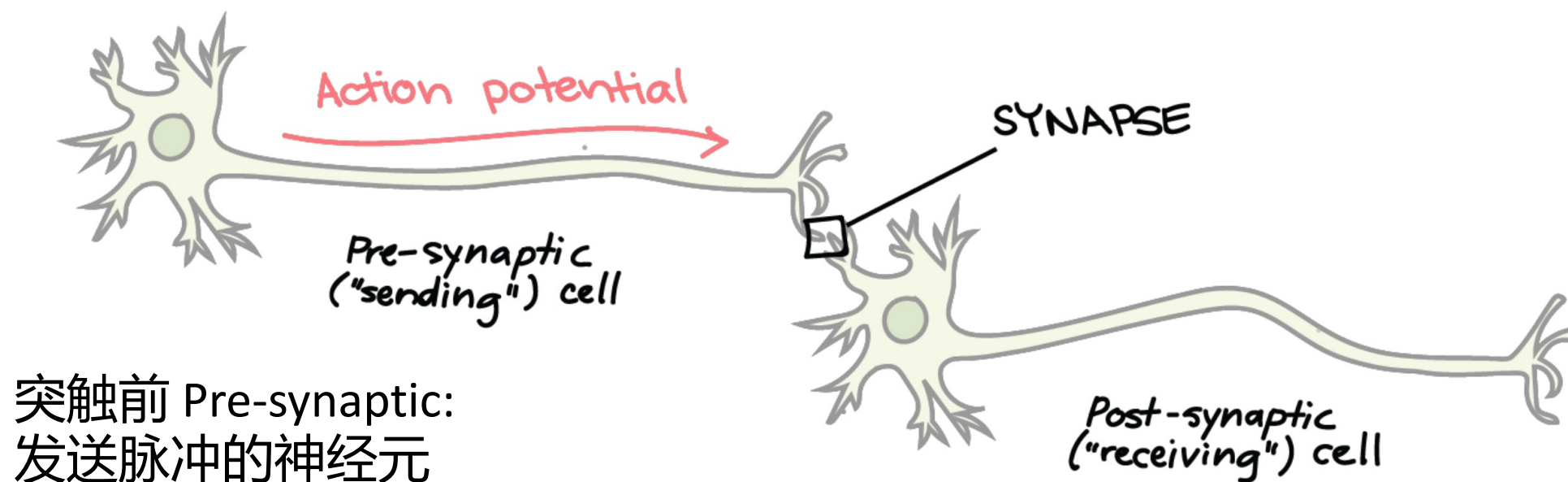
Neuromorphic Computing

- NMC: 神经形态计算/类脑计算
- 生物神经元 => 大脑中的神经细胞
- 神经元 => 人工神经元: 可用软件模拟, 或用硬件实现
- 生物神经元相互交流的方法: **脉冲 (spikes)** / 动作电位 / 神经冲动
 - 大脑中约有870亿个生物神经元相互连接, 形成复杂的网络。
 - 脉冲是一种短暂的 (约 2 毫秒) 电脉冲, 通过化学信号 (神经递质) 在神经元之间传递。

Spiking Neural Networks

- 脉冲神经网络 spiking neural networks

- 多个突触前神经元可能连接到一个突触后神经元。
- 突触后神经元也可以连接到突触前神经元，形成环路。



突触前 Pre-synaptic:
发送脉冲的神经元

突触后 Post-synaptic:
接收脉冲的神经元

When and how do neurons spike?

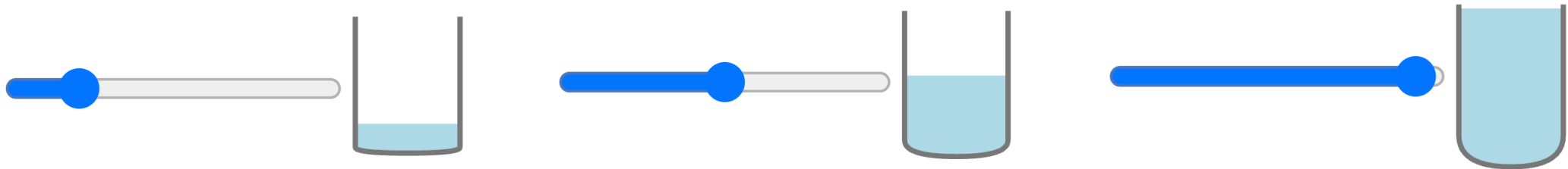
- We can use a simple set of rules to determine when neurons spike.
- A neuron builds up potential energy when it receives pre-synaptic neuron spikes.
- When the potential energy exceeds a threshold, the neuron spikes.
- **Leaky Integrate-and-Fire (LIF)** neurons 渗漏积分激发 模型。

Leaky Integrate-and-Fire (LIF) Neurons

- The post-synaptic neuron adds up (**integrates**) the inputs from all the pre-synaptic neurons but its value decreases (**leaks**) over time.
- If the sum of those inputs reaches a minimum threshold, it **fires**.
- Not a precise simulation but captures the important functional aspects of biological neurons.
- “All models are wrong but some are useful”

LIF: A Leaky Bucket Analogy

- Imagine the neuron as a bucket with a bottom made of rubber.
- The more water we add, the higher “potential” the neuron has.



- Full bucket => the neuron is close to firing.

Source: <https://soney.github.io/snn-from-scratch/chapters/03 - Basic Neuron Models.html>

LIF: A Leaky Bucket Analogy

- Imagine we poke a small hole in the rubber bottom, then the water will slowly leak out of the role.
- If we add more water, the hole will expand and let out water faster.
- An analogy to biological neurons, whose the membrane potential decreases over time without additional inputs.



Source: <https://soney.github.io/snn-from-scratch/chapters/03 - Basic Neuron Models.html>

LIF: Mathematical Notations

- Refer to the potential as v (for voltage), which changes over time as a function of time t : $v(t)$
- Derivative $v'(t)$: the rate of change of $v(t)$ at time t . (the speed at which water leaks out of bucket)
 - $v'(t) < 0$: we are losing water; $v'(t) > 0$: adding water.
- $v'(t)$ is determined by a few factors:
 - $I(t)$: the amount of incoming water at time t
 - $v(t)$: current amount of water (water leaks out faster if the bucket is more full)
 - τ : how the hole grows in relation to the amount of water (how “stretchy” the water is)

LIF: Mathematical Notations

- Use a formula:

$$v'(t) = \frac{1}{\tau} I(t) - \frac{1}{\tau} v(t)$$

- If $I(t) = 0$: no incoming water, then

$$v'(t) = -\frac{1}{\tau} v(t)$$

- The water leaks out
slower for larger values of τ ;
faster for smaller values.

$\tau = 500$:



$\tau = 1000$:



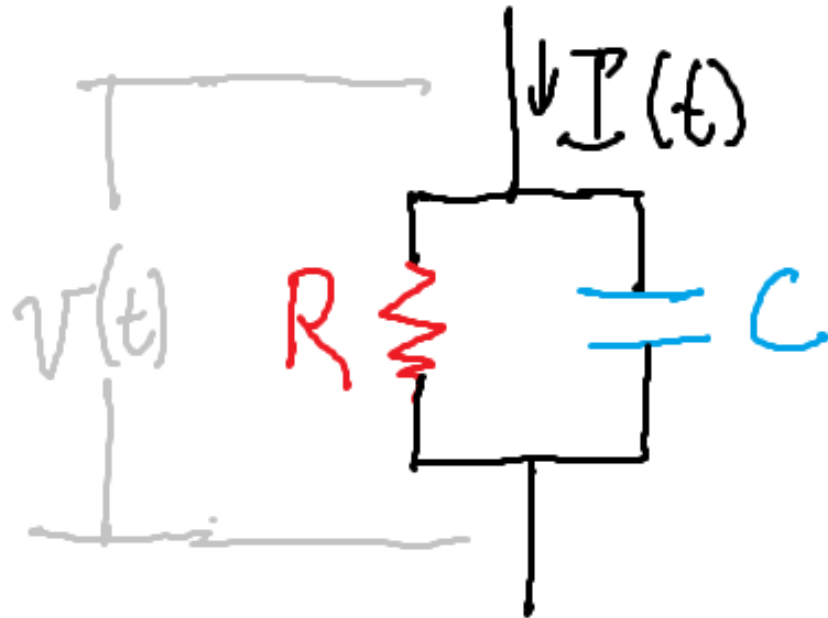
$\tau = 3000$:



Source: <https://soney.github.io/snn-from-scratch/chapters/03 - Basic Neuron Models.html>

Resistor-Capacitor (RC) circuit model

- A more “standard” way of introducing LIF model



R : resistance of a resistor;

C : capacitance of a capacitor

$I(t) = I_R(t) + I_C(t)$: the amount of current that flows through the resistor and capacitor

Ohm's law: $I_R(t) = \frac{v(t)}{R}$

For the capacitor: $I_C(t) = C v'(t)$

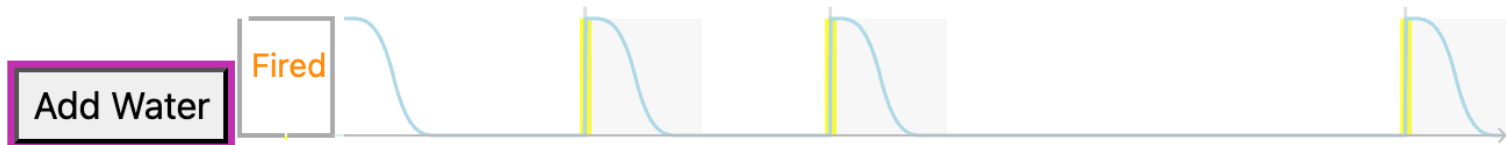
Source: <https://soney.github.io/snn-from-scratch/chapters/03 - Basic Neuron Models.html>

Resistor-Capacitor (RC) circuit model (cont.)

- Thereby, $I(t) = I_R(t) + I_C(t) = \frac{v(t)}{R} + C v'(t)$
- $\Rightarrow v'(t) = \frac{1}{C} I(t) - \frac{1}{RC} v(t)$
- Let $I_2(t) = RI(t)$, and $\tau_{RC} = RC$, then: $v'(t) = \frac{1}{\tau_{RC}} I_2(t) - \frac{1}{\tau_{RC}} v(t)$
- Recall the previous generic formula:
 - $v'(t) = \frac{1}{\tau} I(t) - \frac{1}{\tau} v(t)$. Now τ is replaced with τ_{RC} with a physical meaning.

LIF: Firing

- Once the potential reaches a firing threshold v_{th} , the neuron **fires** (triggers an action potential 动作电位).
- Threshold: the neuron will only fire when $v(t) \geq v_{th}$.
- After firing, then neuron enters a “refractory period” (不应期) where its potential resets and it cannot fire.



Source: <https://soney.github.io/snn-from-scratch/chapters/03 - Basic Neuron Models.html>

First-Order Approximation of LIF

- In continuous time, $v'(t) = \frac{1}{\tau_{RC}}I(t) - \frac{1}{\tau_{RC}}v(t)$ * we remove the subscript in the current term $I_2(t)$
- Using Euler's approximation method, given $v(t)$ and its rate of change $v'(t)$, we can estimate $v(t + \Delta t)$:
$$v(t + \Delta t) \approx v(t) + v'(t)\Delta t$$
- Substitute $v'(t)$:

$$v(t + \Delta t) \approx v(t) + \left(\frac{1}{\tau_{RC}}I(t) - \frac{1}{\tau_{RC}}v(t) \right) \Delta t$$

$$v(t + \Delta t) \approx v(t) \left(1 - \frac{\Delta t}{\tau_{RC}} \right) + \frac{\Delta t}{\tau_{RC}}I(t)$$

Discrete Time Simulation

- Let $v[t]$ be an approximation of $v(t)$: $v[t] \approx v\left(\frac{1}{T_{step}}t\right)$,
where T_{step} is the minimal time step (small enough for getting an accurate Euler's approximation)
- Then we have the discrete form:
$$v[t + 1] \approx v[t] \left(1 - \frac{T_{step}}{\tau_{RC}}\right) + \frac{T_{step}}{\tau_{RC}} I(t)$$
- which can be implemented iteratively:
- $v[0] = v_{init}$
- $$v[t] = v[t - 1] \left(1 - \frac{T_{step}}{\tau_{RC}}\right) + \frac{T_{step}}{\tau_{RC}} I(t)$$

* easier to implement by replacing $I(t - 1)$ with $I(t)$

Python Implementation of LIF

Source: <https://soney.github.io/snn-from-scratch/chapters/04 - First Order LI.html>

```
class FirstOrderLI: # First Order Leaky Integrate
    def __init__(self, tau_rc=0.2, v_init=0): # Default values for tau_rc and v_init
        self.tau_rc = tau_rc # Set instance variables
        self.v = v_init

    def step(self, I, t_step): # Advance one time step (input I and time step size t_step)
        self.v =
```

$$v[t] = v[t - 1] \left(1 - \frac{T_{step}}{\tau_{RC}} \right) + \frac{T_{step}}{\tau_{RC}} I(t)$$

- The potential $v[t]$ is stored in `self.v`
- Every time `step()` is called, the LIF model moves forward by one time step and updates `self.v`

Implementing Firing

- To implement the firing mechanism, we need to set the threshold potential and the refractory period.
- v_{th} : if the neuron's potential reaches this threshold, it will fire.
- τ_{ref} : after the neuron fires, it needs to wait this long until it “accepts” input again.
- Represent firing as the neuron's “output”
 - By default the output will be 0
 - When firing, the output will be $1/T_{step}$ (so that the total energy within one time step will be the same regardless of step length)

Python Implementation of LIF firing

Source: <https://soney.github.io/snn-from-scratch/chapters/04 - First Order LI.html>

```
class FirstOrderLIF: # First Order Leaky Integrate and Fire
    def __init__(self, tau_rc=0.2, tau_ref=0.002, v_init=0, v_th=1):
        self.tau_rc = tau_rc # Potential decay time constant
        self.v = v_init # Potential value
        self.v_th = v_th # Firing threshold
        self.tau_ref = tau_ref # Refractory period time constant

        self.output = 0 # Current output value
        self.refractory_time = 0 # Current refractory period time (how long until the neuron can fire again)

    def step(self, I, t_step): # Advance one time step (input I and time step size t_step)
        self.refractory_time -= t_step # Subtract the amount of time that passed from our refractory time

        if self.refractory_time < 0: # If the neuron is not in its refractory period
            self.v = self.v - (self.v - 0) * self.tau_rc * t_step # Integrate input and leak
            self.v = self.v + I * t_step # Add input current

            if self.v > self.v_th: # If the potential is above the threshold
                self.refractory_time = self.tau_ref # Enter the refractory period
                self.output = 1 # Emit a spike
                self.v = 0 # Reset the potential
            else: # If the potential is below the threshold
                self.output = 0 # Do not fire

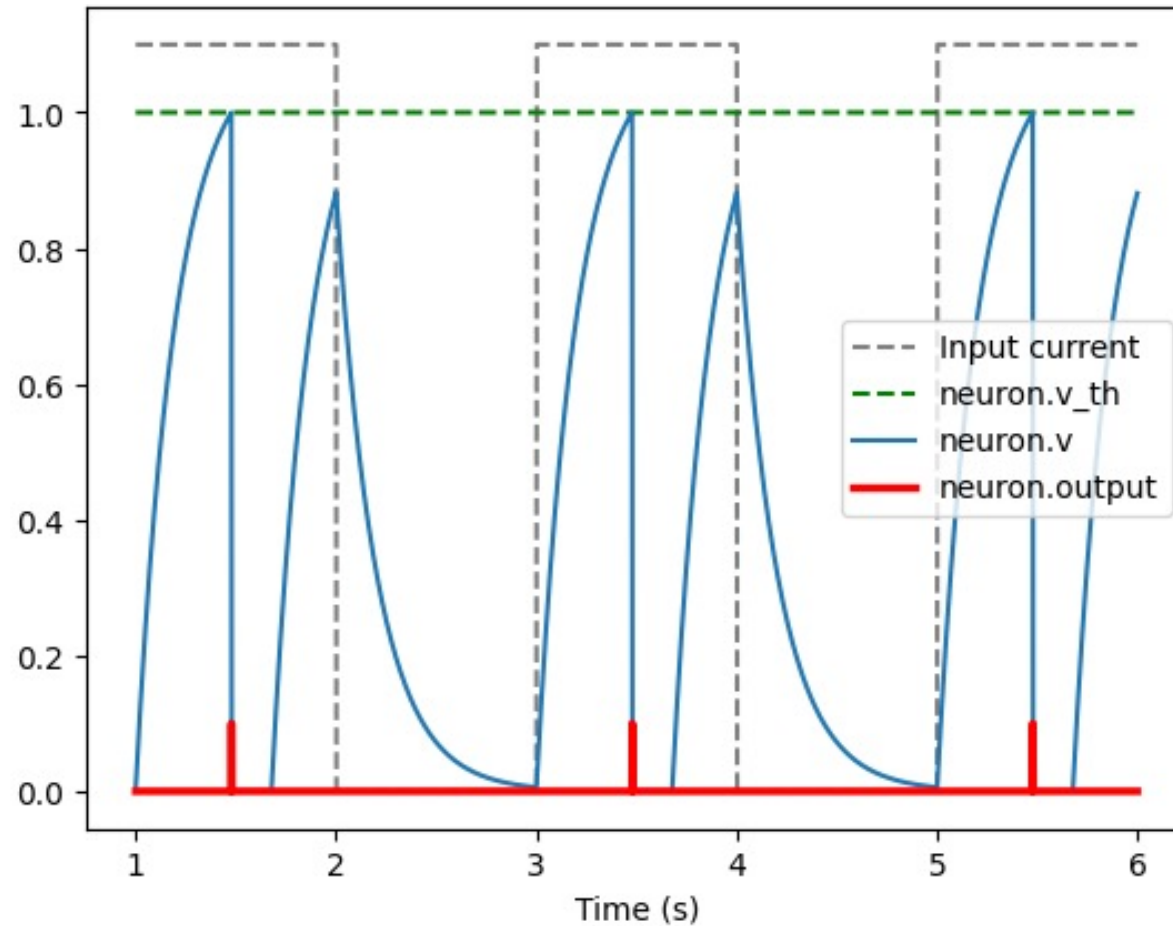
        return self.output
```

→ outside refractory time

} fire and enter refractory time

Firing output

Source: <https://soney.github.io/snn-from-scratch/chapters/04 - First Order LI.html>



Synaptic Models

- So far, we have created neurons but haven't actually connected them. We need a synaptic model (突触模型) to achieve this.
- A synaptic model can “convert” binary spikes into smoother potentials that decay exponentially over time.
- Synaptic model behaves similar to LIF model, except that rather than being binary (spike or not), the output will be the potential itself, defined by function $g(t)$.
- Start at $g(t) = 0$. Whenever there is a spike, $g(t)$ will instantaneously increment by the spike's potential. Otherwise, it will have a rate of change:

$$g'(t) = -\frac{1}{\tau_s} g(t)$$

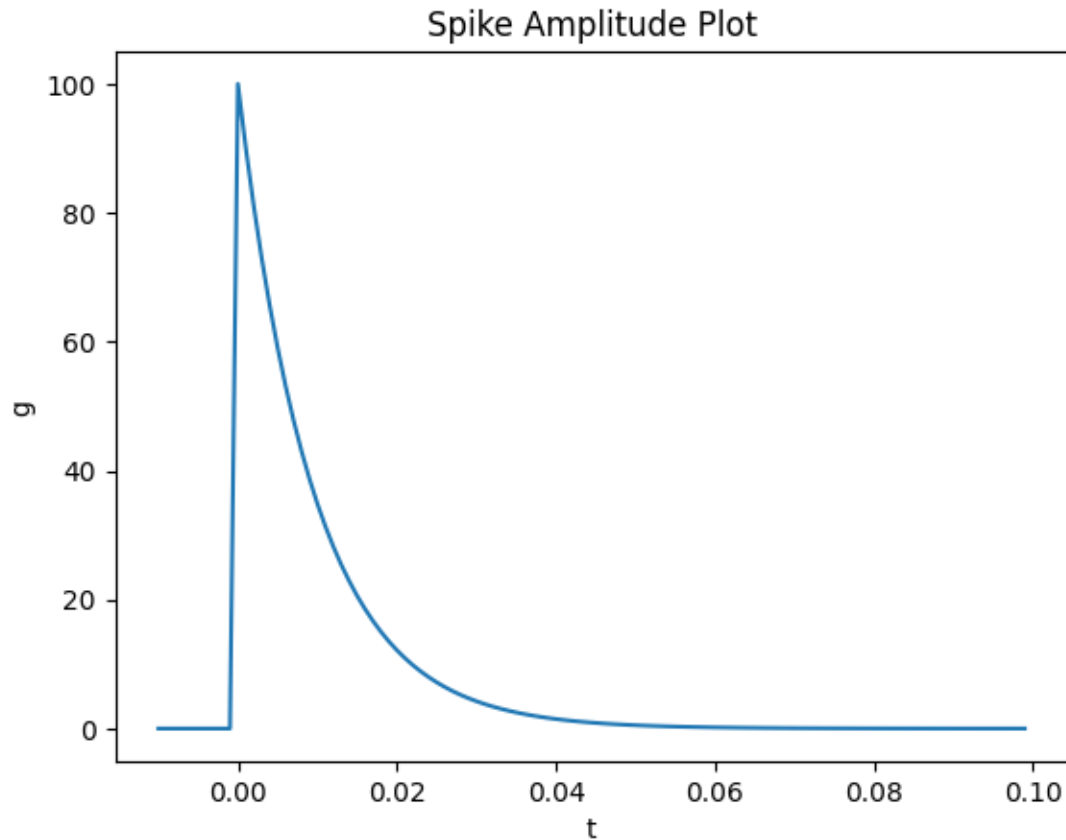
Synaptic Models

$$g'(t) = -\frac{1}{\tau_s} g(t)$$

- It is analogous to the equation of $v'(t)$ for LIF neuron, except with no incoming current.
- τ_s determines how quickly $g(t)$ decays back to 0. (similar to τ_{RC} for $v'(t)$)
- Similar to what we did to LIF, we can approximate its value as:
 - $g[0] = 0$
 - $g[t] = g[t - 1] \left(1 - \frac{T_{step}}{\tau_s}\right) + \frac{T_{step}}{\tau_s} \delta_s[t]$
- Where $\delta_s[t]$ represents the potential of any incoming spikes (or 0 if there are no incoming spikes).

Synaptic Models

Source: <https://soney.github.io/snn-from-scratch/chapters/07 - Synapses.html>



```
import matplotlib.pyplot as plt
import numpy as np

tau_s = 0.01
t_step = 0.001

times = np.arange(-0.01, 0.1, t_step)

def isZero(t): return abs(t) < t_step / 2 # check if t is close to zero

g = 0
g_values = []
for t in times:
    if isZero(t): spike_input = 1 / t_step # Spike input at t = 0
    else:         spike_input = 0

    g = g * (1 - t_step / tau_s) + spike_input * t_step / tau_s
    g_values.append(g)
```

Connecting LIF to Synapse

Source: <https://soney.github.io/snn-from-scratch/chapters/07 - Synapses.html>

```
class FirstOrderSynapse:
    def __init__(self, tau_s=0.01):
        self.tau_s = tau_s # Synaptic time constant
        self.output = 0    # Current potential

    def step(self, I, t_step):
        self.output = self.output * (1 - t_step / self.tau_s) + I * t_step / self.tau_s # De
        return self.output
```

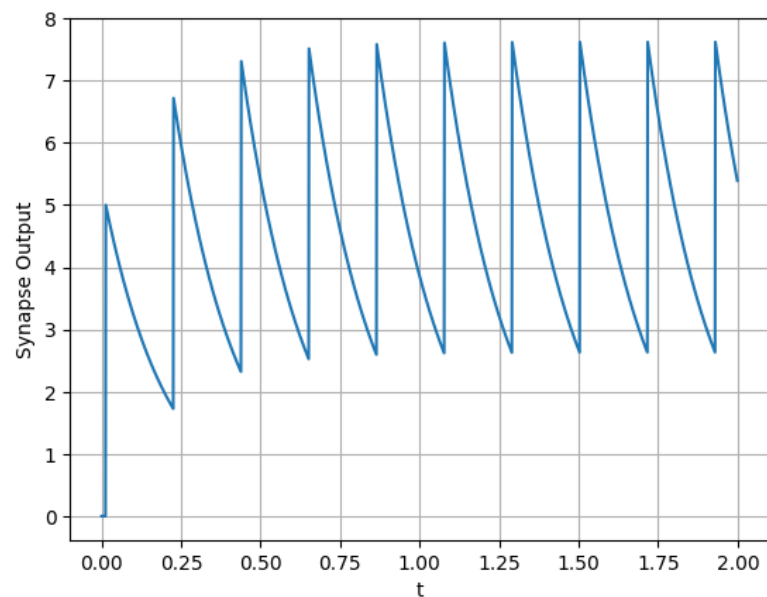
```
neuron = FirstOrderLIF(tau_rc=0.02, tau_ref=0.2)
synapse = FirstOrderSynapse(tau_s=0.2)

def combinedStep(I, t_step):
    neuron_output = neuron.step(I, t_step)
    synapse_output = synapse.step(neuron_output, t_step)
    return synapse_output
```

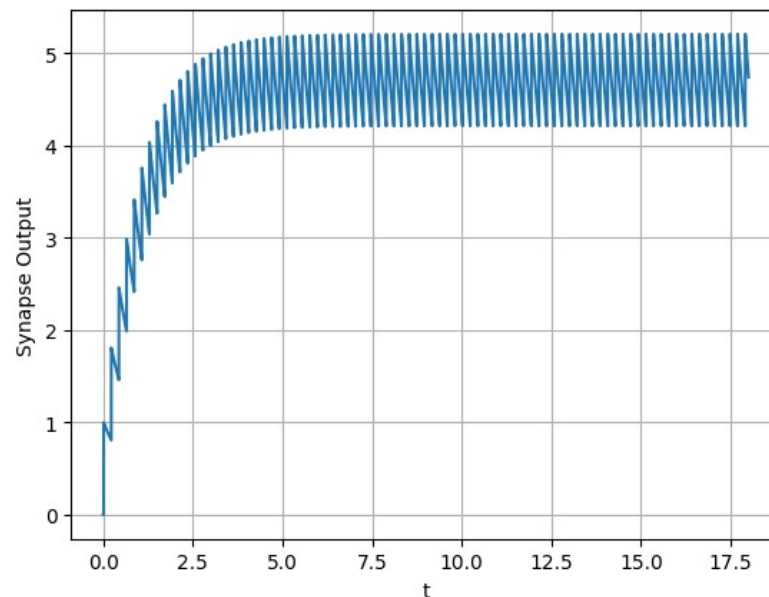
LIF neuron's output is used as input to synapse

Connecting LIF to Synapse

Source: <https://soney.github.io/snn-from-scratch/chapters/07 - Synapses.html>



Small τ_S



Large τ_S

Synapse now has the “memory” of pre-synaptic neuron’s firing history

Adaptive LIF Neurons

- It can also be useful to allow the threshold voltage v_{th} to vary.
- Specifically, when there is a spike, we can slightly increase v_{th} so that it is slightly more difficult to spike next time, adapting to the level of input -- **Adaptive LIF (ALIF)**.
 - It allows the neurons to “fine-tune” their behavior to adapt to the input that might otherwise be too “intense” (and thus resulting in too high of a firing rate).
 - One neuron in a group doesn’t get “greedy” for inputs with which it is closely aligned.

ALIF Implementation

- To implement ALIF, we use an additional variable h to represent how much the threshold voltage v_{th} has increased -- it will only spike when $v \geq v_{th} + h$.
- h will start as 0 and increase by b (strength of adaption) every time it fires.
- So now, instead of a static threshold v_{th} , we have a dynamic threshold v_{th} :

$$v_{th}(t) = v_{th} + h(t)$$

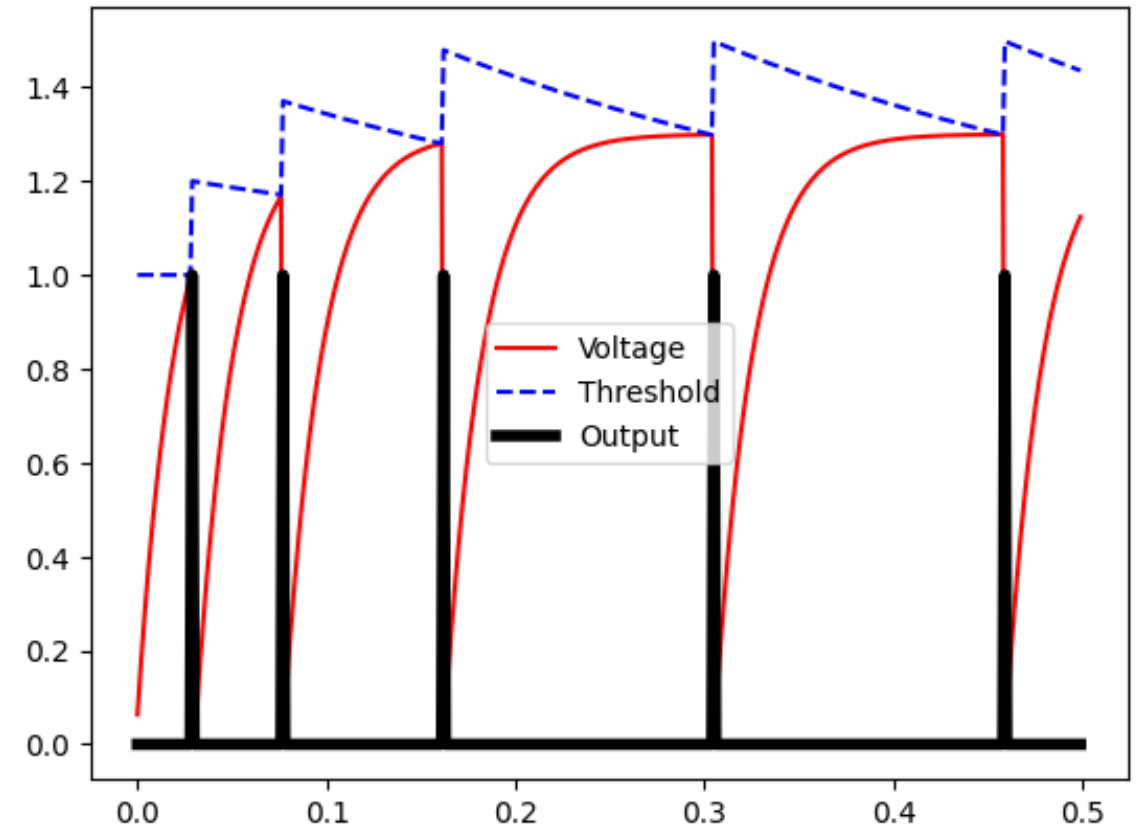
- $h(t)$ decays over time by:

$$h'(t) = -\frac{1}{\tau_w} h(t)$$

ALIF Neurons

Source: <https://soney.github.io/snn-from-scratch/chapters/15 - Adaptive LIF Neurons.html>

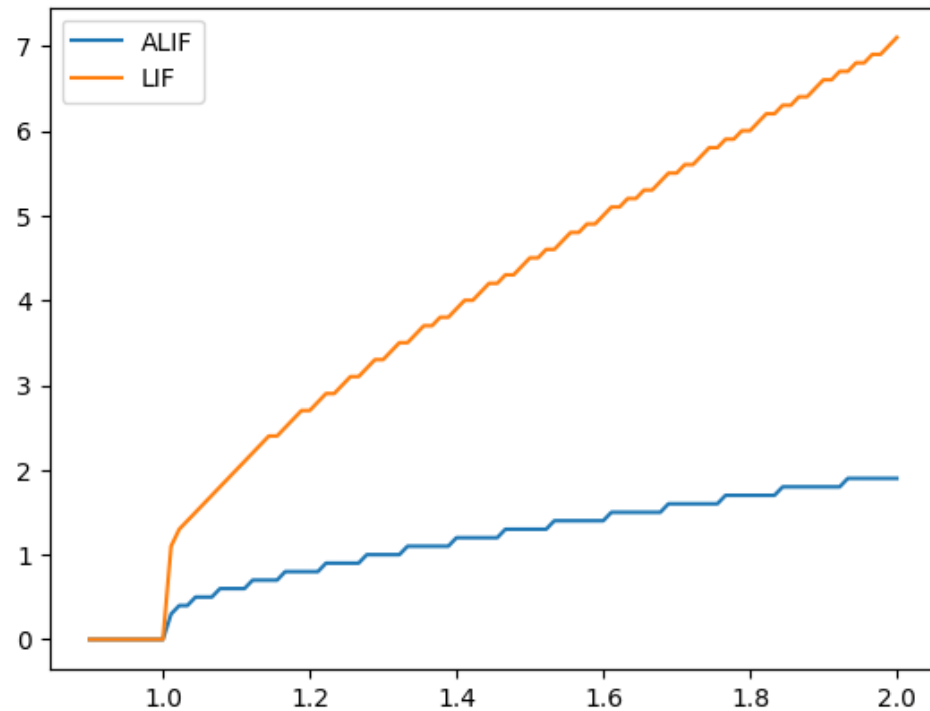
- An ALIF spikes quickly at first, but then the spikes become more spaced over time.
- Firing rate becomes more regular over the long term, as the threshold and voltage balances.



ALIF Neurons

Source: <https://soney.github.io/snn-from-scratch/chapters/15 - Adaptive LIF Neurons.html>

- ALIF (blue) is less sensitive to changes in input than the normal LIF.



Learning through STDP

- Spike-Timing Dependent Plasticity (STDP)
- Review: Hebbian Learning (赫布学习) -- When two neurons fire at similar times, any synaptic connections between them will be strengthened -- “neurons that fire together wire together”.
- Given neurons **A** and **B**, and a connection from A to B ($A \rightarrow B$)
- Then this connection would be **strengthened** if **B fires immediately after A**
However, if **B fires immediately before A**, then the connection will be **weaken**.

STDP

- STDP is a family of update rules for implementing Hebbian learning.
- Focusing on two neurons, “pre” and “post”, which fire at times t_{pre} and t_{post} .
- If $t_{pre} < t_{post}$, then the connection between them should be **strengthened**.
- If $t_{pre} \geq t_{post}$, then the connection should be weakened.
- Let w be the weight between “pre” and “post”. If w is large then “post” will be more likely to fire when “pre” fires; and vice versa. (w can be negative)
- Question: **How much** the weight w should change based on the values of t_{pre} and t_{post} ? I.e., $\Delta w = ?$

STDP Update Rules: $\Delta w = ?$

- Case: $t_{pre} < t_{post}$
- If “post” fires *immediately* after “pre”, then it is likely that “pre” caused “post” to fire, so Δw should be large.
- If “post” fires *long time* after “pre” then it’s not likely that “pre” caused “post” to fire, so Δw should be closer to 0.
- Case: $t_{pre} \geq t_{post}$, Δw will be negative
- The closer t_{pre} is to t_{post} , the more negative Δw should be, since we are more sure that “pre” did **not** cause “post” to fire.

STDP Update Rules

- The effect of “pre” on “post” decays exponentially with time, so Δw should depend on the exponential difference between t_{pre} and t_{post} : $e^{t_{pre}-t_{post}}$.
- Use constants τ_+ and τ_- to represent how Δw decays to 0.

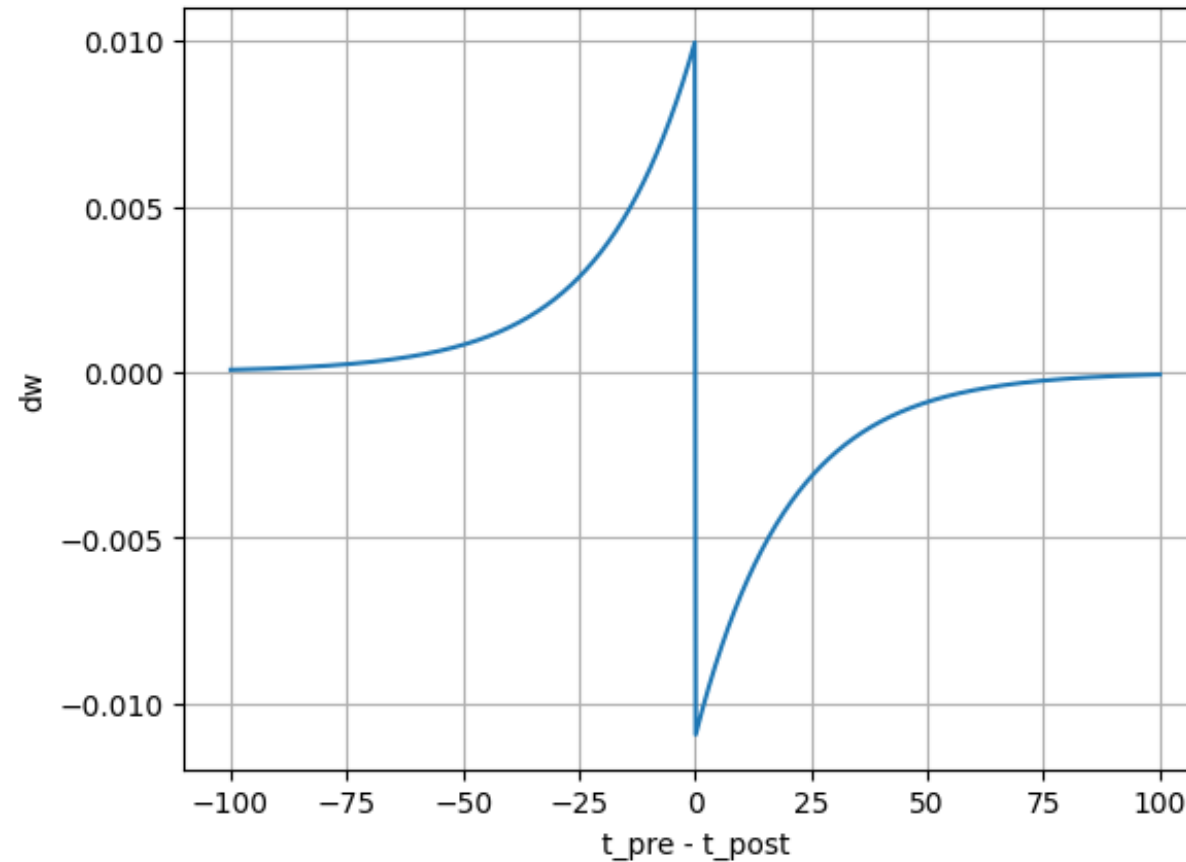
$$\Delta w = \begin{cases} A_+ e^{\frac{(t_{pre}-t_{post})}{\tau_+}}, & \text{if } t_{pre} < t_{post} \\ -A_- e^{\frac{-(t_{pre}-t_{post})}{\tau_-}}, & \text{if } t_{pre} \geq t_{post} \end{cases}$$

- where A_+ and A_- are constants to scale Δw .

STDP Update Rules

Source: <https://soney.github.io/snn-from-scratch/chapters/16-STDP.html>

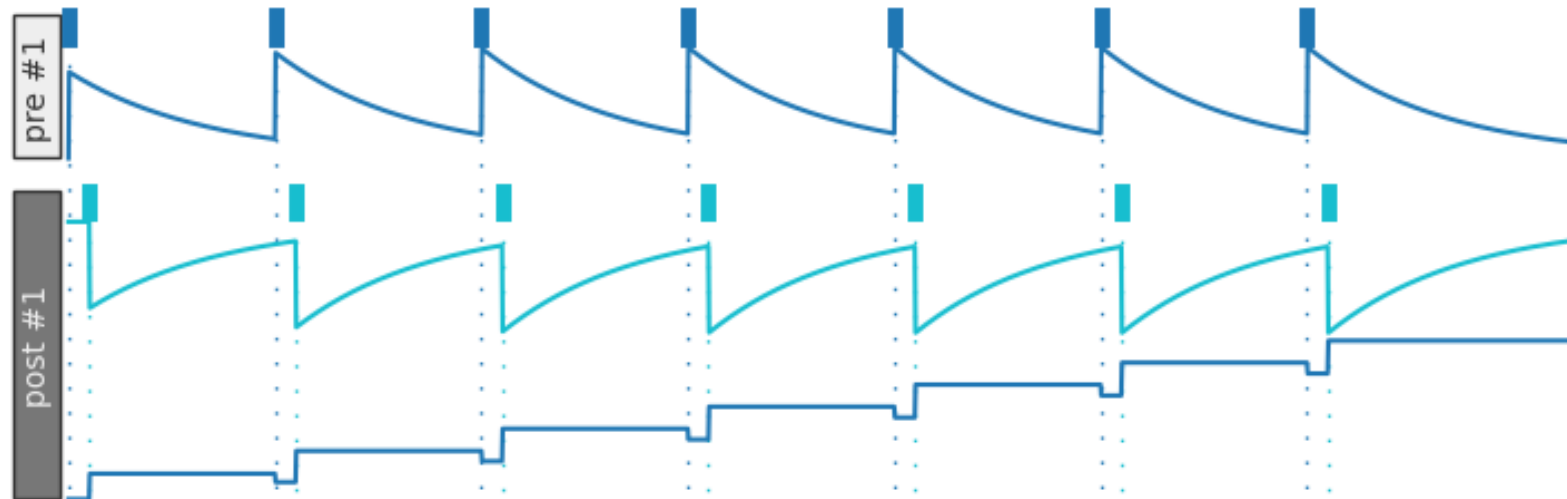
- Plot Δw



STDP demo

Source: <https://soney.github.io/snn-from-scratch/chapters/16-STDP.html>

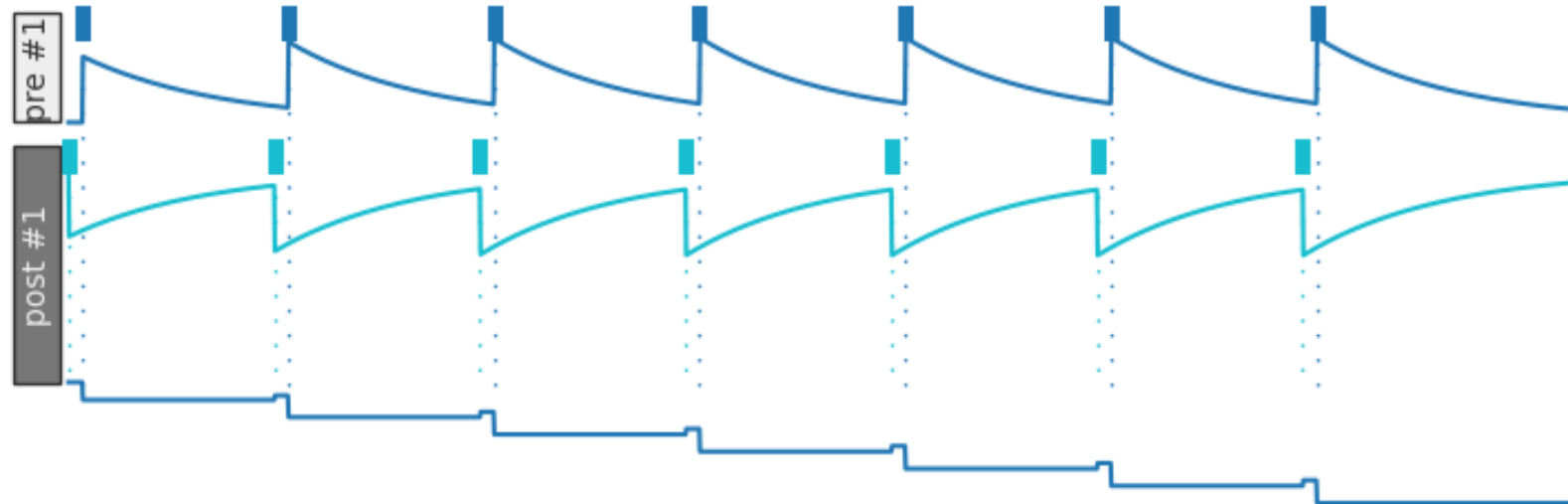
- If the “post” neuron tends to fire *right after* the “pre” neuron, then the connection between the two strengthens.



STDP demo

Source: <https://soney.github.io/snn-from-scratch/chapters/16-STDP.html>

- If the “post” neuron tends to fire *right before* the “pre” neuron, then the connection between the two weakens.



Pros and of SNN/NMC

- **Energy efficiency:** Potentially much more energy-efficient than traditional ANNs, especially when run on *neuromorphic hardware*. Event-driven computation.
- **Biological plausibility:** More closely modeled on how biological neurons function.
- **Temporal processing:** Can process and understand temporal data more effectively than standard ANNs.
- **Compatibility with single-bit signals:** They can work with single-bit signals, which can lead to high efficiency with virtually no loss of accuracy.

Cons and of SNN/NMC

- **Training challenges:** It is difficult to train SNNs using standard methods like backpropagation because the "spikes" are non-differentiable.
- **Lower accuracy:** On some tasks, SNNs can have lower accuracy compared to established ANNs.
- **Computational complexity:** Some SNNs can be computationally complex and require significant resources.
- **Hardware dependency:** SNNs often require specialized neuromorphic hardware to achieve their full energy efficiency potential.

NMC Hardware

Source: <https://open-neuromorphic.org/neuromorphic-computing/hardware/>

- **Memristor (忆阻器):** A resistor with memory, a key component in neuromorphic hardware that can store and process information, mimicking the synaptic plasticity found in biological systems.

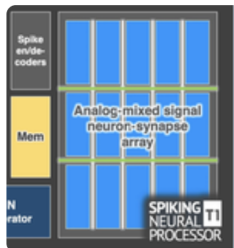


Pulsar - Innatera

Mixed-signal

Released

Pulsar is a brain-inspired neuromorphic microcontroller designed for ultra-low-power, real-time intelligence in edge devices. It combines a Spiking Neural Network (SNN) engine with a RISC-V MCU and CNN acceleration to enable smart sensing applications without cloud dependency.



Spiking Neural Processor T1 by Innatera

Mixed-signal

Released

The Spiking Neural Processor T1 is Innatera's ultra-low-power neuromorphic microcontroller SoC for real-time intelligence close to the sensor. It integrates a spiking neural network accelerator, a convolutional neural network accelerator and a RISC-V core. T1 targets applications in battery-powered, power-limited and latency-critical devices.

Recap

- Spiking Neural Networks (SNN)
- Leaky Integrate-and-Fire (LIF)
 - Bucket of water analogy
 - Circuit model
 - Discrete approximation
- Spike-Timing Dependent Plasticity (STDP)
 - An implementation of Hebbian Learning